NAVSAHYADRI
EDUCATION SOCIETY'S GROUP OF INSTITUTIONS
DEPARTMENT OF COMPUTER ENGINEERING
ACADEMIC YEAR: 2025-26 SEMESTER- II

Web Development
Lab Manual
SE Computer Semester -II

2025-26

Curriculum
Third Year Engineering (2019 Pattern) in
Computer Engineering

Prepared by -
Prof. P.N.Nikam

Institute Vision & Mission

Vision.:.

The institute's vision is to establish the center for excellence in professional development and entrepreneurship development resulting in the enhancement of rural areas. The institute also has a vision for developing professionals and citizens/citizenship to foster professional and rural development.

Mission:

NESGI offers skill-based programs, graduate and post-graduate programmers in engineering and management to build competent manpower to suit the ever-changing requirements in industry and business by supporting students for continual development through excellence, technology-based instructions and overall development of personality in all domains. The institute provides industry-based education and practical training to the rural base.

Departmental Vision & Mission

Vision:

Contributing to the welfare of society through technical and quality education to the rural base students.

Mission:

The Department vision is to establish the center for excellence in professional development and entrepreneurship development resulting in the enhancement of the rural area.

To produce Best Quality Computer Science Professionals by importing quality hands-on experience and value education. To train the students to fulfill Professional and social requirements in the industry.

NAVSAHYADRI
EDUCATION SOCIETY'S GROUP OF INSTITUTIONS
DEPARTMENT OF COMPUTER ENGINEERING
ACADEMIC YEAR: 2025-26 SEMESTER- II

Program Specific Outcomes (PSO)

A graduate of the Computer Engineering Program will demonstrate-

PSOl	Professional Skills-The ability to understand, analyze and develop computer programs in the areas related to algorithms, system software, multimedia, web design, big data analytics, and networking for efficient design of computer-based systems of varying complexities.
PS02	Problem-Solving Skills- The ability to apply standard practices and strategies in software project development using open-ended programming environments to deliver a quality product for business success.
PS03	Successful Career and Entrepreneurship- The ability to employ modern computer languages, environments and platforms in creating innovative career paths to be an entrepreneur and to have a zest for higher studies.

**NAVSAHYADRI
EDUCATION SOCIETY'S GROUP OF INSTITUTIONS
DEPARTMENT OF COMPUTER ENGINEERING ACADEMIC
YEAR: 2025-26 SEMESTER- II**

Program Outcomes (POs)

PO1	**Engineering Knowledge**	Apply knowledge of mathematics, natural sciences, computing, engineering fundamentals, and an engineering specialization (WK1–WK4) to develop solutions for complex engineering problems.
PO2	**Problem Analysis**	Identify, formulate, review research literature, and analyze complex engineering problems to reach substantiated conclusions with consideration for sustainable development (WK1–WK4)
PO3	**Design / Development of Solutions**	Design creative solutions for complex engineering problems and develop systems, components, or processes that meet identified needs with consideration for public health, safety, whole-life cost, net zero carbon, culture, society, and environment (WK5).
PO4	**Conduct Investigations of Complex Problems**	Conduct investigations using research-based knowledge including design of experiments, modelling, analysis, and interpretation of data to provide valid conclusions (WK8).
PO5	**Engineering Tool Usage**	Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling, recognizing their limitations (WK2, WK6).
PO6	**The Engineer and the World**	Analyze and evaluate societal, environmental, and sustainability impacts of engineering solutions with reference to economy, health, safety, legal framework, culture, and environment (WK1, WK5, WK7).
PO7	**Ethics**	Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national and international laws (WK9).
PO8	**Individual and Collaborative Team Work**	Function effectively as an individual and as a member or leader in diverse and multidisciplinary teams.

PO9	**Communication**	Communicate effectively and inclusively within the engineering community and society, including writing reports, design documentation, and delivering effective presentations considering cultural and language differences.
PO10	**Project Management and Finance**	Apply engineering management principles and economic decision-making to manage projects as a member or leader in multidisciplinary environments.
PO11	**Life-Long Learning**	Recognize the need for and engage in independent and life-long learning, adaptability to new and emerging technologies, and critical thinking in the context of technological change.

Course Objectives:

- The course aims to:
- 1. Understand Internet basics, including protocols, client-server architecture, and network security essentials.
- 2. Develop structured web pages using HTML, CSS, and Bootstrap for responsive front-end design.
- 3. Implement interactivity with JavaScript and DOM manipulation techniques.
- 4. Build dynamic web applications using PHP for back-end logic and server-side processing.

Course Outcomes:

Upon successful completion of this course, students will be able to:

• CO1: Explain the fundamentals of Internet architecture, protocols, and client-server interactions.

• CO2: Design responsive web pages using HTML, CSS, and Bootstrap frameworks

• CO3: Apply JavaScript and DOM to create dynamic and interactive web content

• CO4: Develop server-side functionality using PHP for dynamic content generation and form handling

The CO-PO Mapping Matrix for WD practical

CO \ PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	3	2	–	–	1	1	–	–	2	–	2
CO2	2	1	3	–	3	–	–	2	3	–	2
CO3	2	2	3	1	3	–	–	2	3	–	2
CO4	3	2	3	2	3	1	–	2	2	1	3

NAVSAHYADRI
EDUCATION SOCIETY'S GROUP OF INSTITUTIONS
DEPARTMENT OF COMPUTER ENGINEERING
ACADEMIC YEAR: 2025-26 SEMESTER- II

EXPERIMENT NO. 1

Aim: Create a simple HTML page displaying personal details using text formatting tags.

Objective: To design a simple HTML web page that displays personal information using text formatting and heading tags. Course Outcomes Addressed

Outcome: **Students** will successfully create and view a basic web page that neatly presents personal details using HTML text formatting tags

Software and Hardware Requirement:

- Operating System: Windows / Linux
- **Software:** Browser (Chrome/Firefox), VS Code/Notepad++, XAMPP (for PHP)

Theory:

HTML (Hyper Text Markup Language) is used to create web pages. Text formatting tags such as <b>, <i>, <u>, <strong>, and <em> help in emphasizing content and improving readability. Heading tags define the importance and structure of content on a web page.

Formatting elements were designed to display special types of text:

- <b> - **Bold text:** The HTML <b> element defines bold text, without any extra importance
- <strong> - **Important text:** HTML <strong> element defines text with strong importance. The content inside is typically displayed in bold
- <i> - **Italic text:** The HTML <i> element defines a part of text in an alternate voice or mood. The content inside is typically displayed in italic.
- <em> - **Emphasized text:** The HTML <em> element defines emphasized text. The content inside is typically displayed in italic.
- <mark> - Marked text: The HTML <mark> element defines text that should be marked or highlighted:
- <small> - **Smaller text:** The HTML <small> element defines smaller text:
- <del> - **Deleted text:** The HTML <del> element defines text that has been deleted from a document. Browsers will usually strike a line through deleted text:
- <ins> - **Inserted text**: The HTML <ins> element defines a text that has been inserted into a document. Browsers will usually underline inserted text:
- <sub> - **Subscript text**: The HTML <sub> element defines subscript text. Subscript text appears half a character below the normal line, and is sometimes rendered in a smaller font. Subscript text can be used for chemical formulas, like H2O:
- <sup> - **Superscript text**: The HTML <sup> element defines superscript text. Superscript text appears half a character above the normal line, and is sometimes rendered in a smaller font. Superscript text can be used for footnotes, like WWW[1]:
-

Procedure:

1. Open a text editor.
2. Write the basic HTML structure.
3. Add personal details using formatting tags.
4. Save the file with .html extension.
5. Open it in a web browser

Code:

```
<!DOCTYPE>
<html>
<head>
 <title>PersonalDetails</title>
</head>
<body>
<h1>PersonalDetails</h1>
<p><b>Name:</b>StudentName</p>
<p><i>Department:</i>ComputerEngineering</p>
<p><u>College:</u>XYZEngineeringCollege</p>
<p><strong>Email:</strong>student@gmail.com</p>
<p><em>ContactNo:</em>9876543210</p>
</body>
</html>
```

Output: A web page displaying personal details with formatted text.

Conclusion: Thus, a simple HTML web page was successfully created using text formatting tags.

Viva Questions:

1. What is HTML?

2. What are text formatting tags?

3. Difference between <b> and <strong> tags?

4. Which tag is used for italics?

5. What is the use of heading tags?

EXPERIMENT NO. 2

Aim: Design a web page that includes an image, hyperlink, and a nested table.

Objective: To understand the use of basic HTML tags.

To insert and display an image on a web page.

To create and use hyperlinks for navigation.

To design tables and nested tables using HTML.

To gain practical exposure to web page design concepts.

Outcome: After completing this experiment, students will be able to:

- Create a basic HTML web page.
- Insert images using the <img> tag.
- Create hyperlinks using the <a> tag.
- Design tables and nested tables using <table>, <tr>, and <td> tags.
- Understand the layout and structure of HTML documents.

Software and Hardware Requirement:

● Operating System: Windows / Linux● Software: Browser (Chrome/Firefox), VS Code/Notepad++, XAMPP (for PHP)

Theory: HTML (Hyper Text Markup Language) is the standard language used to create web pages. It uses tags and attributes to define the structure and presentation of content on the web.

- Image: Images are added to a web page using the <img> tag. The src attribute specifies the image path, and alt provides alternative text.

The HTML <img> tag is used to embed an image in a web page.

Images are not technically inserted into a web page; images are linked to web pages. The <img> tag creates a holding space for the referenced image.

The <img> tag is empty, it contains attributes only, and does not have a closing tag.

The <img> tag has two required attributes:

- src - Specifies the path to the image
- alt - Specifies an alternate text for the image
    -
    - Hyperlink: Hyperlinks are created using the <a> tag, which allows navigation from one page to another or to external websites.
    - The HTML <a> tag defines a hyperlink. It has the following syntax:
    - The most important attribute of the <a> element is the href attribute, which indicates the link's destination.
    - The *link text* is the part that will be visible to the reader.
    - Clicking on the link text, will send the reader to the specified URL address
    -
    - Table: Tables are used to display data in rows and columns using <table>, <tr>, <th>, and <td> tags.HTML tables allow web developers to arrange data into rows and columns.
    - Each table cell is defined by a <td> and a </td> tag.
    - Everything between <td> and </td> is the content of a table cell.
    - Each table row starts with a <tr> and ends with a </tr> tag.
    - Sometimes you want your cells to be table header cells. In those cases, use the <th> tag instead of the <td> tag:
    - By default, the text in <th> elements are bold and centered, but you can change that with CSS.
    - Nested Table: A table placed inside another table is called a nested table. It is useful for displaying complex or grouped data.

Procedure:

Open any text editor (Notepad, VS Code, etc.).

Create a new file and save it with .html extension (e.g., index.html).

Write the basic HTML structure using <html>, <head>, and <body> tags.

Insert an image using the <img> tag.

Add a hyperlink using the <a> tag.

Create a table and insert another table inside one of its cells (nested table).

Save the file and open it in a web browser.

Observe the output displayed in the browser.

Code:

```
<!DOCTYPE html>

<html>

<head>

  <title>Image, Hyperlink and Nested Table</title>

</head>

<body>

<h1>My Web Page</h1>

<!-- Image -->

<h2>Image</h2>

<img src="image.jpg" alt="Sample Image" width="200" height="150">

<!-- Hyperlink -->

<h2>Hyperlink</h2>
```

```
 <a href="https://www.google.com" target="_blank">Click here to visit Google</a>

<!-- Table with Nested Table -->

<h2>Table with Nested Table</h2>

<table border="1" cellpadding="5">

   <tr>

      <th>Name</th>

       <th>Details</th>

   </tr>

   <tr>

       <td>Amit</td>

       <td>

           <table border="1" cellpadding="5">

               <tr>

                   <th>Subject</th>

                   <th>Marks</th>

               </tr>

               <tr>

                   <td>Maths</td>

                   <td>80</td>

               </tr>

               <tr>
```

```
            <td>Science</td>

            <td>85</td>

          </tr>

        </table>

      </td>

    </tr>

  </table>

</body>

</html>
```

Output

- A web page displaying a heading.
- An image displayed on the page.
- A clickable hyperlink that opens Google in a new tab.
- A table containing student information with a nested table showing subject-wise marks.

Conclusion: Thus, a web page containing an image, a hyperlink, and a nested table was successfully designed using HTML. This experiment helped in understanding the basic building blocks of web page design.

Viva Questions

1. What is HTML?
2. Which tag is used to insert an image in HTML?
3. What is the use of the <a> tag?
4. What is a nested table?
5. Which attribute is used to specify the image path?
6. What is the difference between <th> and <td>?
7. Why is the alt attribute important in the <img> tag?
8. What does the target="_blank" attribute do?

EXPERIMENT NO. 3

Aim: To create an HTML form with fields: Name, Email, Gender, Date of Birth, and Submit button.

Objective:

- To design a basic HTML form
- To collect user information such as name, email, gender, and date of birth
- To understand different HTML form input types
- To use a submit button to send form data

Outcome: After completing this experiment, the student will be able to:

- Create an HTML form using the <form> tag
- Use input types like text, email, radio, and date
- Create radio buttons for gender selection
- Design a simple and user-friendly web form

Software and Hardware Requirement:

● Operating System: Windows / Linux● Software: Browser (Chrome/Firefox), VS Code/Notepad++, XAMPP (for PHP)

Theory: An HTML form is used to collect user input and send it to a server for processing.

Forms are created using the <form> tag.

Common Form Elements

- <input type="text"> – used for name
- <input type="email"> – used for email (validates format)
- <input type="radio"> – used to select gender
- <input type="date"> – used to select date of birth
- <input type="submit"> – used to submit the form

The required attribute ensures that the user must fill the field before submitting the form.

The <input type="text"> defines a single-line input field for text input.

- The <label> Element
- Notice the use of the <label> element in the example above.
- The <label> tag defines a label for many form elements.
- The <label> element is useful for screen-reader users, because the screen-reader will read out loud the label when the user focuses on the input element.
- The <label> element also helps users who have difficulty clicking on very small regions (such as radio buttons or checkboxes) - because when the user clicks the text within the <label> element, it toggles the radio button/checkbox.
- The for attribute of the <label> tag should be equal to the id attribute of the <input> element to bind them together.

- The <input type="radio"> defines a radio button.
- The <input type="submit"> defines a button for submitting the form data to a form-handler.
- The form-handler is typically a file on the server with a script for processing input data.
- The form-handler is specified in the form's action attribute.
- Notice that each input field must have a name attribute to be submitted.
- If the name attribute is omitted, the value of the input field will not be sent at all.

  Radio buttons let a user select ONE of a limited number of choices.
- The <input type="checkbox"> defines a **checkbox**.

  Checkboxes let a user select ZERO or MORE options of a limited number of choices

Procedure:

- Open **Notepad**
- Type the HTML code for creating the form
- Save the file with .html extension (example: form.html)
- Open the file in any web browser
- Fill the form fields and click the submit button

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Registration Form</title>
</head>
<body>
  <h2>Registration Form</h2>
  <form>
    <!-- Name -->
    <label>Name:</label><br>
    <input type="text" name="name" required><br><br>

    <!-- Email -->
    <label>Email:</label><br>
    <input type="email" name="email" required><br><br>

    <!-- Gender -->
    <label>Gender:</label><br>
    <input type="radio" name="gender"> Male
    <input type="radio" name="gender"> Female
    <input type="radio" name="gender"> Other
    <br><br>

   <!-- Date of Birth -->
    <label>Date of Birth:</label><br>
```

```
        <input type="date" name="dob"><br><br>
        <!-- Submit Button -->
        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

Output

- A registration form is displayed on the web page
- The form contains fields for name, email, gender, and date of birth
- Gender is selected using radio buttons
- A submit button is displayed at the bottom of the form
- On clicking submit, the form data is sent

Conclusion: Thus, an HTML form with fields name, email, gender, date of birth, and submit button was successfully created using basic HTML form elements.

VIVA Questions & Answers

1. **What is an HTML form?**
   An HTML form is used to collect user input and send it to a server.
2. **Which tag is used to create a form?**
   <form> tag.
3. **Which input type is used for entering name?**
   type="text"
4. **Which input type is used for email?**
   type="email"
5. **What is the advantage of using type="email"?**
   It validates the email format automatically.
6. **Which input type is used for selecting gender?**

   type="radio"

7. **Why is the same name attribute used in radio buttons?**

   To allow selection of only one option at a time.

8. **Which input type is used for date of birth?**

   type="date"

9. **What is the use of submit button?**

   To send form data to the server.

10. **Which input type is used for submit button?**

    type="submit"

11. **What is the use of the required attribute?**

    It makes a field mandatory.

12. **Which tag is used to label form elements?**

    <label> tag.

13. **What happens when the submit button is clicked?**

    The form data is submitted to the server.

14. **Can a form work without a server?**

    Yes, but the data will not be processed.

15. **What is the default method of form submission?**

    GET method.

EXPERIMENT NO. 4

Aim: To design a web page and apply **inline CSS, internal CSS, and external CSS** to style headings and tables.

Objective:

- To understand the concept of CSS
- To learn different types of CSS
- To apply inline CSS to individual elements
- To apply internal CSS using the <style> tag
- To apply external CSS using a separate CSS file

Outcome: After completing this experiment, the student will be able to:

- Differentiate between inline, internal, and external CSS
- Style headings using different CSS methods
- Design and format tables using CSS
- Create a structured and attractive web page

Software and Hardware Requirement:

● Operating System: Windows / Linux● Software: Browser (Chrome/Firefox), VS Code/Notepad++, XAMPP (for PHP)

Theory: Cascading Style Sheets (CSS) CSS is used to control the presentation and layout of HTML web pages. It allows separation of content (HTML) from design (CSS).

CSS saves a lot of work. It can control the layout of multiple web pages all at once.

CSS can be added to HTML documents in 3 ways:

- **Inline** - by using the style attribute inside HTML elements
- **Internal** - by using a <style> element in the <head> section
- **External** - by using a <link> element to link to an external CSS file

The most common way to add CSS, is to keep the styles in external CSS files. However, in this

tutorial we will use inline and internal styles, because this is easier to demonstrate, and easier for you to try it yourself.

Types of CSS

1. Inline CSS

- An inline CSS is used to apply a unique style to a single HTML element.
- An inline CSS uses the style attribute of an HTML element.
- The following example sets the text color of the <h1> element to blue, and the text color of the <p> element to red:
- <h1 style="color:blue;">A Blue Heading</h1>
  <p style="color:red;">A red paragraph.</p>
- Inline CSS is written directly inside an HTML tag using the style attribute. It affects only that specific element and has the highest priority.
- Example: <h1 style="color: green;">Heading</h1>

2. Internal CSS

3. External CSS

External CSS is written in a separate .css file and linked to the HTML page using the <link> tag. It is best for **large websites** and reusable styles.

An external style sheet is used to define the style for many HTML pages.

To use an external style sheet, add a link to it in the <head> section of each HTML page:

```
        <html>
<head>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
</html>
```

Procedure:

1. Open **Notepad**
2. Create an HTML file and save it as index.html

3. Create a CSS file and save it as style.css
4. Link the external CSS file in the HTML <head> section
5. Apply inline CSS to one heading
6. Apply internal CSS to table elements
7. Apply external CSS to remaining headings and table
8. Open the HTML file in a web browse

Code:

External CSS File (style.css)

```
        /* External CSS */
h2 {
   color: blue;
   text-align: center;
   font-family: Arial;
}

table {
   width: 60%;
   margin: auto;
```

border-collapse: collapInternal CSS is written inside the <style> tag in the <head> section of the HTML document. It applies to that **single web page**.

- An internal CSS is used to define a style for a single HTML page.
- An internal CSS is defined in the <head> section of an HTML page, within a <style> element.
- The following example sets the text color of ALL the <h1> elements (on that page) to blue, and the text color of ALL the <p> elements to red. In addition, the page will be displayed with a "powderblue" background color:
- <html>

```
<head>
<style>
body {background-color: powderblue;}
```

```
h1   {color: blue;}
p    {color: red;}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
</html>


se;
}
```

HTML File (index.html)

```
<!DOCTYPE html>
<html>
<head>
   <title>CSS Types Practical</title>
   <!-- External CSS Link -->
   <link rel="stylesheet" href="style.css">
   <!-- Internal CSS -->
   <style>
      th {
         background-color: lightgray;
         padding: 10px;
      }
      td {
         text-align: center;
         padding: 8px;
      }

      table, th, td {
```

```
            border: 1px solid black;
        }
    </style>
</head>
<body>

    <!-- Inline CSS -->
    <h1 style="color: green; text-align: center;">
        Student Details
    </h1>
    <!-- External CSS -->
    <h2>Department Information</h2>

    <table>
        <tr>
            <th>Roll No</th>
            <th>Name</th>
            <th>Course</th>
        </tr>
        <tr>
            <td>1</td>
            <td>Pooja</td>
            <td>BSc CS</td>
        </tr>
        <tr>
            <td>2</td>
            <td>Rahul</td>
            <td>BSc IT</td>
        </tr>
    </table>
</body>
</html>
```

Output

- A green colored main heading aligned at the center (inline CSS)
- A blue colored subheading aligned at the center (external CSS)
- A table with borders, padding, and background color (internal CSS)
- The web page is properly styled using all three types of CSS

Conclusion

Thus, inline, internal, and external CSS were successfully applied to style headings and tables in a web page.

VIVA Questions with Answers

1. **What is CSS?**

   CSS (Cascading Style Sheets) is used to style and format HTML web pages.

2. **How many types of CSS are there?**

   There are **three types**: Inline CSS, Internal CSS, and External CSS.

3. **What is inline CSS?**

   Inline CSS is written inside an HTML tag using the style attribute.

4. **Which CSS has the highest priority?**

   Inline CSS has the highest priority.

5. **What is internal CSS?**

   Internal CSS is written inside the <style> tag in the <head> section of an HTML page.

6. **What is external CSS?**

   External CSS is written in a separate .css file and linked to the HTML page.

7. **Which tag is used to link external CSS?**

   <link> tag.

8. **Why is external CSS preferred for large websites?**

   Because it allows reusability and easy maintenance.

9. **What is the use of CSS in tables?**

   CSS is used to add borders, padding, colors, and alignment to tables.

10. **What does border-collapse do?**

    It merges table borders into a single border.

11. **Which CSS property is used to change text color?**

    color

12. **Which property is used to align text in the center?**

    text-align

13. **Can we use all three types of CSS in one web page?**

    Yes, all three types can be used together.

14. **What happens if same style is applied in different CSS types?**

    The style with higher priority (inline > internal > external) is applied.

15. **Why is CSS called "Cascading"?**

    Because styles are applied based on priority order.

EXPERIMENT NO. 5

Aim: Develop a responsive web page using Bootstrap Grid and Components.

Objective:

To study the concept of **responsive web design**.

To understand the **Bootstrap framework** and its advantages.

To learn the usage of **Bootstrap Grid System** for creating responsive layouts.

To implement various **Bootstrap components** such as navbar, buttons, cards, and forms.

To design a web page that adapts automatically to different screen sizes (mobile, tablet, desktop).

Outcome: After completing this experiment, students will be able to:

Apply **Bootstrap CSS and JavaScript** in web development.

Design **responsive layouts** using the Bootstrap grid system.

Use **predefined Bootstrap components** to enhance UI design.

Create web pages that are **device-independent and user-friendly**.

Develop modern, responsive web applications efficiently.

Software and Hardware Requirement:

- Operating System: Windows / Linux
- Software: Browser (Chrome/Firefox), VS Code/Notepad++, XAMPP (for PHP)

Theory: Responsive Web Page using Bootstrap Grid and Components Responsive Web Design (RWD) is an approach to web development that ensures web pages render well on a variety of devices and screen sizes such as desktops, tablets, and mobile phones. It uses flexible layouts, grids, and media queries to adapt content dynamically. **Bootstrap** is an open-source front-end framework developed by Twitter. It provides pre-designed CSS and JavaScript components that help developers create responsive and mobile-first websites quickly and efficiently. Bootstrap follows a **mobile-first approach**, meaning styles are optimized for smaller screens first and then scaled up for larger devices.

Bootstrap Grid System

The Bootstrap Grid System is a flexible layout system based on a **12-column structure**. It allows developers to divide the web page into rows and columns for better alignment and responsiveness.

Container: Used to wrap site content (.container or .container-fluid).

Row: Used to create horizontal groups of columns.

Columns: Defined using classes such as .col-sm-*, .col-md-*, .col-lg-*, etc.The grid automatically adjusts layout based on screen size.

Example:

- col-sm → Small devices (≥576px)
- col-md → Medium devices (≥768px)
- col-lg → Large devices (≥992px)
- col-xl → Extra-large devices (≥1200px)

Bootstrap Components

Bootstrap provides reusable UI components that enhance the look and functionality web pages without writing custom CSS.

Common Bootstrap components include:

- **Navbar** – Used for responsive navigation menus.
- **Buttons** – Pre-styled buttons with different colors and sizes.
- **Cards** – Used to display content such as images, text, and links.
- **Forms** – Styled form controls for user input.
- **Alerts** – Used to display messages or notifications.

Procedure:

1.Create a new folder for the experiment and open a text editor (VS Code / Notepad++).

2.Create a new HTML file and save it with a .html extension.

3.Write the basic HTML5 structure using <!DOCTYPE html>, <html>, <head>, and <body> tags.

4.Include the **Bootstrap CSS and JavaScript CDN links** inside the <head> section of the HTML file.

5.Add a Bootstrap **container (.container or .container-fluid)** to hold the web page content.

6.Create a layout using the **Bootstrap Grid System** by adding .row and .col-* classes.

7.Insert responsive **Bootstrap components** such as navbar, buttons, cards, and forms inside the grid columns.

8.Apply appropriate Bootstrap utility classes for spacing, alignment, and colors.

9.Save the HTML file and open it in a web browser.

10.Resize the browser window or use developer tools to verify the responsiveness of the web page on different screen sizes.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Bootstrap Responsive Web Page</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
<!-- Navbar Component -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" href="#">MyWebsite</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#menu">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="menu">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link" href="#">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="#">About</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Services</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Contact</a></li>
      </ul>
    </div>
```

```
    </div>
</nav>
<!-- Header Section -->
<div class="container-fluid bg-primary text-white text-center p-5">
    <h1>Welcome to Bootstrap</h1>
    <p>Responsive Web Page using Grid & Components</p>
    <button class="btn btn-light">Learn More</button>
</div>
<!-- Grid Section -->
<div class="container mt-5">
    <div class="row">
        <!-- Card 1 -->
        <div class="col-sm-12 col-md-6 col-lg-4 mb-4">
            <div class="card">
                <div class="card-body">
                    <h5 class="card-title">Web Design</h5>
                    <p class="card-text">Creating responsive and modern websites.</p>
                    <a href="#" class="btn btn-primary">Read More</a>
                </div>
            </div>
        </div>
        <!-- Card 2 -->
        <div class="col-sm-12 col-md-6 col-lg-4 mb-4">
            <div class="card">
                <div class="card-body">
                    <h5 class="card-title">Development</h5>
                    <p class="card-text">Building fast and scalable applications.</p>
                    <a href="#" class="btn btn-success">Read More</a>
                </div>
            </div>
        </div>
        <!-- Card 3 -->
```

```
      <div class="col-sm-12 col-md-6 col-lg-4 mb-4">
        <div class="card">
          <div class="card-body">
            <h5 class="card-title">SEO</h5>
            <p class="card-text">Optimizing websites for search engines.</p>
            <a href="#" class="btn btn-warning">Read More</a>
          </div>
        </div>
      </div>
    </div>
</div>
<!-- Footer -->
<footer class="bg-dark text-white text-center p-3 mt-5">
</footer>
<!-- Bootstrap JS -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Output

- A responsive web page is successfully developed using Bootstrap Grid System and Components.
  The web page displays a responsive navigation bar, content arranged using the 12-column grid layout, and various Bootstrap components such as cards, buttons, and forms.
- The layout automatically adjusts according to different screen sizes including desktop, tablet, and mobile devices without breaking the design.

Conclusion

Hence, the responsive web page is successfully developed using the Bootstrap Grid System and Components. The experiment helped in understanding the mobile-first approach of Bootstrap and the effective use of the 12-column grid layout.

Viva Questions

1. What is responsive web design?
2. What is Bootstrap?
3. Why is Bootstrap used in web development?
4. What is meant by mobile-first approach in Bootstrap?
5. Explain the Bootstrap Grid System.
6. How many columns are there in Bootstrap grid?
7. What is the purpose of .container and .container-fluid?
8. What is a row in Bootstrap grid system?

EXPERIMENT NO. 6

Aim: Write a JavaScript program to validate form inputs (e.g., email, empty fields).

Objective:

To understand and implement client-side form validation using JavaScript.

To validate user inputs such as empty fields and email format before form submission.

To improve data accuracy and enhance user experience by preventing invalid input.

Outcome:

Successfully validated form inputs using JavaScript.

Ensured mandatory fields are not left empty.

Verified the correctness of email format using regular expressions.

Gained practical knowledge of JavaScript event handling and form validation techniques.

Software and Hardware Requirement:

● Operating System: Windows / Linux

Software: Browser (Chrome/Firefox), VS Code/Notepad++, XAMPP (for PHP)

Theory:

JavaScript Form Validation: Form validation is the process of verifying whether the data entered by a user in a form is correct, complete, and in the required format before it is submitted to the server. JavaScript form validation is a type of client-side validation, which is performed in the user's web browser before the form data is sent for further processing.

Need for Form Validation

- To ensure that mandatory fields such as name and email are not left empty.
- To check whether the input data follows a specific format (for example, a valid email address).
- To prevent incorrect or incomplete data from being submitted.
- To improve user experience by providing immediate feedback.
- To reduce unnecessary server load by catching errors on the client side.

Role of JavaScript in Form Validation

JavaScript provides built-in functions and methods to access form elements, read user input, and apply logical conditions. Using JavaScript:

- Form fields can be accessed using methods like getElementById() or getElementsByName().
- Conditions such as checking for empty values can be applied using if statements.

  Regular expressions (RegEx) can be used to validate patterns like email addresses.
- Alerts or messages can be displayed to guide the user to correct input errors.

Email Validation

Email validation ensures that the entered email address follows a standard format such as:username@domain.com

JavaScript uses regular expressions to check whether the email contains:

- A valid username
- The @ symbol
- A valid domain name

Empty Field Validation

Empty field validation checks whether required input fields are left blank. If any required field is empty, JavaScript prevents the form from being submitted and displays an error message, prompting the user to fill in the missing information.

Procedure:

JavaScript Form Validation

1. Create an HTML Form

   Design a basic HTML form with input fields such as *Name* and *Email* using <input> elements.
2. Assign Unique IDs to Form Fields

   Provide unique id attributes to each input field so that they can be accessed easily using JavaScript.
3. Create a JavaScript Validation Function

   Write a JavaScript function (for example, validate Form()) that will be called when the form is submitted.
4. Access Form Input Values Use JavaScript methods like document.getElementById() to retrieve the values entered by the user.

5. Check for Empty Fields
6. Apply conditional statements (if) to verify whether required fields such as name and email are left empty.
7. Validate Email Format Use a regular expression (RegEx) to check whether the entered email address follows a valid email format.
8. Display Error Messages If validation fails, display appropriate alert messages to inform the user about the error.
9. Prevent Form Submission on Error

   Return false from the validation function to stop the form from being submitted when invalid data is detected.
10. Allow Form Submission on Success If all input values are valid, return true to allow successful form submission.
11. Test the Form Run the web page in a browser and test different input cases to verify that validation works correctly.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Form Validation</title>
  <script>
    function validateForm() {
      // Get form values
      var name = document.forms["myForm"]["name"].value;
      var email = document.forms["myForm"]["email"].value;
      // Check empty fields
      if (name == "" || email == "") {
        alert("All fields are required!");
        return false;
      }
      // Email validation using Regular Expression
      var emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

```
            if (!emailPattern.test(email)) {
                alert("Please enter a valid email address!");
                return false;
            }
            alert("Form submitted successfully!");
            return true;
        }
    </script>
</head>
<body>
    <h2>Student Registration Form</h2>
    <form name="myForm" onsubmit="return validateForm()">
        Name: <input type="text" name="name"><br><br>
        Email: <input type="text" name="email"><br><br>

        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

Output

Case 1: Empty Fields

- When the user clicks Submit without entering any data:
- Alert Message: "Name field cannot be empty"
- Form submission is stopped.

Case 2: Invalid Email Format

- When the user enters an incorrect email format:
- Alert Message: "Please enter a valid email address"
- Form submission is prevented.

Case 3: Valid Input

- When the user enters a valid name and email:

- Alert Message: "Form submitted successfully!"

- Form is submitted successfully.

Conclusion: Thus, the JavaScript program for validating form inputs was successfully implemented. The program effectively checks for empty fields and verifies the correctness of the email format before form submission.

Viva Questions

1. What is form validation?
2. Why is JavaScript used for form validation?
3. What is client-side validation?
4. How do you access form fields in JavaScript?
5. How do you check for empty fields in JavaScript?
6. What is an email validation?
7. What is a regular expression (RegEx)?
8. Why is RegEx used in email validation?

EXPERIMENT NO. 7

Aim: Create a web page that uses JavaScript to display dynamic content using DOM.

Objective:

To understand the concept of the **Document Object Model (DOM)** in JavaScript.

To learn how JavaScript can be used to **dynamically access and modify HTML elements**.

To display and update web page content dynamically based on user interaction.

To improve skills in **client-side scripting** using JavaScript.

Outcome:

Successfully created a web page that displays **dynamic content using JavaScript and DOM**.

Gained practical knowledge of accessing HTML elements using DOM methods such as getElementById().

Learned how to modify content dynamically without reloading the web page.

Understood the role of JavaScript in creating **interactive and responsive web applications**.

Software and Hardware Requirement:

- Operating System: Windows / Linux
- Software: Browser (Chrome/Firefox), VS Code/Notepad++, XAMPP (for PHP)

Theory: Dynamic Content Display Using JavaScript and DOM

The **Document Object Model (DOM)** is a programming interface that represents an HTML document as a structured tree of objects. Each element of a web page, such as headings, paragraphs, buttons, and images, is treated as a node in this tree structure. JavaScript uses the DOM to access, manipulate, and modify the content and structure of a web page dynamically.

Role of JavaScript in DOM Manipulation

JavaScript enables interaction with HTML elements by using DOM methods such as getElementById(), getElementsByClassName(), and querySelector(). Through these methods, JavaScript can read or change the content, attributes, and styles of HTML elements while the page is running, without requiring a page reload.

Dynamic Content

Dynamic content refers to web page content that changes in response to user actions or events such as button clicks, mouse movements, or page loading. By using JavaScript and DOM manipulation, text, images, and other elements can be updated in real time, making web pages more interactive and user-friendly.

Event Handling

Events are actions performed by the user, such as clicking a button or typing in a field. JavaScript can respond to these events using event handlers like onclick. When an event occurs, a JavaScript function is executed, which then modifies the DOM to display updated content.

Advantages of Using DOM for Dynamic Content

- Enables real-time content updates without refreshing the page
- Improves user interaction and responsiveness
- Makes web pages more flexible and interactive
- Reduces server load by handling changes on the client side

Procedure: Displaying Dynamic Content Using JavaScript and DOM

1. 1Create a basic HTML structure using <!DOCTYPE html>, <html>, <head>, and <body> tags.
2. Add required HTML elements such as headings, paragraphs, and buttons that will be modified dynamically.
3. Assign unique id attributes to the HTML elements so they can be accessed using JavaScript.
4. Include a <script> tag either in the <head> section or at the end of the <body> section.
5. Write a JavaScript function to access HTML elements using DOM methods like document.getElementById().
6. Use JavaScript to change the content of HTML elements by modifying properties such as innerHTML or textContent.
7. Attach event handlers (for example, onclick) to buttons or other elements to trigger dynamic changes.
8. Execute the web page in a browser and interact with the elements to observe dynamic content updates.

9. Verify that the content changes without reloading the web page.

10. Save and test the program to ensure correct functioning of DOM manipulation.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Dynamic Content using DOM</title>
  <script>
    function changeContent() {
      // Access element using DOM
      document.getElementById("message").innerHTML =
        "Welcome! Content updated using JavaScript DOM.";
      // Change style dynamically
      document.getElementById("message").style.color = "blue";
      document.getElementById("message").style.fontSize = "24px";
      // Display date and time
      document.getElementById("time").innerHTML =
        "Current Date & Time: " + new Date();
    }
  </script>
</head>
<body>
  <h2>JavaScript DOM Example</h2>
  <p id="message">Click the button to change this text.</p>
  <p id="time"></p>
  <button onclick="changeContent()">Click Me</button>
</body>
</html>
```

Output:

Initial Output

- The web page displays a **heading**, a **paragraph**, and a **button**.

- The paragraph initially shows default text such as:

  "Click the button to change the content."

  After User Interaction

- When the user clicks the **button**, JavaScript is executed.
- The content of the paragraph changes dynamically using DOM manipulation.
- The updated text is displayed instantly, for example:

  "The content has been updated dynamically using JavaScript and DOM."

Conclusion: Thus, a web page using JavaScript and the Document Object Model (DOM) was successfully created to display dynamic content. The program demonstrated how JavaScript can access and modify HTML elements in real time based on user interaction.

Viva Questions

1. What is DOM?
2. Why is DOM used in JavaScript?
3. What is dynamic content?
4. How do you access an HTML element using DOM?
5. What is inner HTML?
6. What is an event in JavaScript?
7. Which event is commonly used to change content dynamically?
8. What is the use of JavaScript in web pages?

EXPERIMENT NO. 8

Aim: Write a JavaScript program for a simple calculator using functions and switch-case.

Objective:

To understand the use of JavaScript functions for modular programming.

To implement switch–case control structure in JavaScript.

To perform basic arithmetic operations such as addition, subtraction, multiplication, and division.

To develop a simple calculator using JavaScript for client-side computation**.**

Outcome:

- Successfully designed a JavaScript program for a simple calculator.
- Gained practical knowledge of using functions to perform calculations.
- Learned how switch–case is used to select arithmetic operations.
- Understood how JavaScript handles user input and output dynamically.
- Enhanced problem-solving skills in client-side scripting.

Software and Hardware Requirement:

- Operating System: Windows / Linux
- Software: Browser (Chrome/Firefox), VS Code/Notepad++, XAMPP (for PHP)

Theory: A calculator is an application used to perform arithmetic operations such as addition, subtraction, multiplication, and division. In JavaScript, a calculator can be implemented using functions and the switch–case control structure.

Functions help in organizing code into reusable blocks, making the program modular and easy to understand.

Switch–case is a decision-making statement used when multiple conditions are based on a single expression. It selects the correct arithmetic operation depending on the user’s choice.

JavaScript performs calculations on the client side, providing quick results without server interaction.

Using functions with switch–case improves code readability, reduces repetition, and makes the program efficient.

Procedure:

1. Create a basic HTML structure using standard HTML tags.
2. Add input fields to accept two numbers from the user.
3. Provide buttons or options to select the arithmetic operation.
4. Write a JavaScript function to perform calculations.
5. Use the switch–case statement inside the function to select the operation.
6. Perform arithmetic operations based on the selected case.
7. Display the calculated result on the web page.
8. Save the file with .html extension.
9. Open the file in a web browser.
10. Test the calculator with different inputs and operations.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Simple Calculator</title>
  <script>
    function calculate() {

      // Read input values
      var num1 = parseFloat(document.getElementById("n1").value);
      var num2 = parseFloat(document.getElementById("n2").value);
      var op = document.getElementById("operator").value;
      var result;
      // Switch case for operations
      switch (op) {
        case "+":
          result = num1 + num2;
          break;
```

```
            case "-":
                result = num1 - num2;
                break;
            case "*":
                result = num1 * num2;
                break;
            case "/":
                if (num2 == 0) {
                    alert("Division by zero not allowed");
                    return;
                }
                result = num1 / num2;
                break;
            default:
                alert("Invalid operator");
                return;
        }
        // Display result
        document.getElementById("result").value = result;
    }
  </script>
</head>
<body>
  <h2>Simple Calculator</h2>
  Number 1:
  <input type="text" id="n1"><br><br>
  Number 2:
  <input type="text" id="n2"><br><br>
  Operator:
  <select id="operator">
      <option value="+">+</option>
      <option value="-">?</option>
```

```
    <option value="*">×</option>
    <option value="/">÷</option>
  </select><br><br>
  <button onclick="calculate()">Calculate</button><br><br>
  Result:
  <input type="text" id="result" readonly>
</body>
</html>
```

Output

Case 1: Addition

- *Input: Number1 = 10, Number2 = 5*
- *Operation: Addition*
- *Output:*
- *Result = 15*

Case 2: Subtraction

- *Input: Number1 = 10, Number2 = 5*
- *Operation: Subtraction*
- *Output:*
- *Result = 5*

Case 3: Multiplication

- *Input: Number1 = 10, Number2 = 5*
- *Operation: Multiplication*
- *Output:*
- *Result = 50*

Case 4: Division

- *Input: Number1 = 10, Number2 = 5*
- *Operation: Division*
- *Output:*

Result = 2

Conclusion: Thus, a simple calculator using JavaScript was successfully implemented with the help of functions and the switch–case statement. The program performs basic arithmetic operations efficiently and demonstrates the use of control structures and modular programming in JavaScript.

Viva Questions

1. What is a function in JavaScript?
2. Why is switch–case used in a calculator program?
3. What are arithmetic operators in JavaScript?
4. What is client-side scripting?
5. What happens if division by zero occurs?
6. What is the advantage of using functions?
7. Can switch–case replace if–else?
8. Give one real-life example of a calculator application.

EXPERIMENT NO. 9

Aim:

To design a simple PHP web page that displays a welcome message along with the current date and time.

Objective:

To understand the use of PHP scripting for dynamic content generation by displaying the current system date and time on a web page.

Course Outcomes Addressed:

- Apply basic PHP syntax to create dynamic web pages
- Display server-side date and time using PHP built-in functions

Outcome:

Students will successfully create and execute a PHP script that displays a welcome message along with the current date and time.

Software and Hardware Requirement:

- Operating System: Windows / Linux
- Software: Browser (Chrome/Firefox), VS Code/Notepad++, XAMPP (for PHP)
- Hardware: Any system capable of running XAMPP

Theory:

PHP (Hypertext Preprocessor) is a server-side scripting language used to develop dynamic and interactive web pages. PHP scripts are executed on the server, and the result is returned to the browser as plain HTML.

PHP provides built-in functions to work with date and time:

- `date()` – Formats a local date and time
- `date_default_timezone_set()` – Sets the default timezone

Common date format characters:

- `d` – Day
- `m` – Month
- `Y` – Year

- `h` – Hour
- `i` – Minutes
- `s` – Seconds
- `A` – AM/PM

Procedure:

1. Install and start the XAMPP server.
2. Open a text editor.
3. Write the PHP script to display a welcome message and current date & time.
4. Save the file with `.php` extension inside the `htdocs` folder.
5. Open a web browser and run the file using `localhost`.

\

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Welcome Page</title>
</head>
<body>

<h1>Welcome</h1>

<?php
date_default_timezone_set("Asia/Kolkata");
echo "Current Date: " . date("d-m-Y") . "<br>";
echo "Current Time: " . date("h:i:s A");
?>

</body>
</html>
```

Output:

A web page displaying a Welcome message along with the current date and time.

Conclusion:

Thus, a PHP script was successfully created to display a welcome message and the current date and time using PHP built-in functions.

Viva Questions:

1. What is PHP?
2. Why is PHP called a server-side scripting language?
3. Which function is used to display date and time in PHP?
4. What is the use of date_default_timezone_set()?
5. Difference between static HTML and PHP web pages?

EXPERIMENT NO. 10

Aim:

To write a PHP program that accepts user input through an HTML form and displays the entered data using the POST method.

Objective:

To understand the working of HTML forms and the POST method in PHP for transferring user data securely.

Course Outcomes Addressed:

- Understand HTML form handling in PHP
- Apply POST method to collect user input
- Develop dynamic and interactive web pages

Outcome:

Students will successfully create and execute a PHP program that accepts form input and displays the submitted data using the POST method.

Software and Hardware Requirement:

- Operating System: Windows / Linux
- Software: Browser (Chrome/Firefox), VS Code/Notepad++, XAMPP (for PHP)
- Hardware: Any system capable of running XAMPP

Theory:

An HTML form is used to collect user input and send it to the server for processing. PHP can collect form data using predefined superglobal arrays.

The POST method:

- Sends data securely
- Does not display data in the URL
- Suitable for sensitive information

PHP superglobal:

- `$_POST` – Collects form data sent using POST method

- ## Procedure:

1. Install and start XAMPP server.
2. Open a text editor.
3. Create an HTML form using POST method.
4. Write PHP code to display submitted data.
5. Save the file with `.php` extension inside `htdocs`.
6. Run the file using `localhost` in a web browser.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>PHP POST Method</title>
</head>
<body>

<h1>Student Information Form</h1>

<form method="post" action="">
    Name: <input type="text" name="name"><br><br>
    Email: <input type="email" name="email"><br><br>
    Contact: <input type="text" name="contact"><br><br>
    <input type="submit" name="submit" value="Submit">
</form>

<?php
if(isset($_POST['submit']))
{
    $name = $_POST['name'];
    $email = $_POST['email'];
    $contact = $_POST['contact'];
```

```
    echo "<h3>Submitted Details:</h3>";
    echo "Name: " . $name . "<br>";
    echo "Email: " . $email . "<br>";
    echo "Contact: " . $contact . "<br>";
}
?>

</body>
</html>
```

Output:

A web page that accepts user input through a form and displays the submitted details after clicking the submit button.

Conclusion:

Thus, a PHP program was successfully written to accept form input and display it using the POST method.

Viva Questions:

1. What is the POST method in PHP?

2. What is the difference between GET and POST methods?

3. What is `$_POST` superglobal?

4. Why is POST method more secure than GET?

5. What is an HTML form?

EXPERIMENT NO. 11

Aim:

To implement a PHP program that performs string manipulation operations such as finding length, reversing a string, and extracting a substring.

Objective:

To understand string handling functions in PHP and apply them for performing basic string manipulation operations.

Course Outcomes Addressed:

- Understand string functions in PHP
- Apply PHP built-in functions for string processing
- Develop dynamic web pages using PHP

Outcome:

Students will successfully create and execute a PHP program that performs string manipulation operations like reverse, length, and substring.

Software and Hardware Requirement:

- **Operating System:** Windows / Linux
- **Software:** Browser (Chrome/Firefox), VS Code/Notepad++, XAMPP (for PHP)
- **Hardware:** Any system capable of running XAMPP

Theory:

A string in PHP is a sequence of characters used to store and manipulate text. PHP provides several built-in functions to perform string operations efficiently.

Common string functions:

- `strlen()` – Returns the length of a string
- `strrev()` – Reverses a string
- `substr()` – Extracts a part of a string

String manipulation is widely used in form validation, data processing, and dynamic content generation.

Procedure:

1. Install and start the XAMPP server.
2. Open a text editor.
3. Write a PHP script for string manipulation operations.

4. Save the file with `.php` extension inside the `htdocs` folder.
5. Run the file using `localhost` in a web browser.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>PHP String Manipulation</title>
</head>
<body>

<h1>String Manipulation in PHP</h1>

<?php
$string = "Computer Engineering";

echo "Original String: " . $string . "<br><br>";

echo "Length of String: " . strlen($string) . "<br>";

echo "Reversed String: " . strrev($string) . "<br>";

echo "Substring (0 to 8): " . substr($string, 0, 8);
?>

</body>
</html>
```

Output:

A web page displaying the original string, its length, reversed string, and substring.

Conclusion:

Thus, a PHP program was successfully implemented to perform string manipulation operations using built-in PHP string functions.

Viva Questions:

1. What is a string in PHP?
2. Which function is used to find string length in PHP?
3. What does `strrev()` do?
4. Explain the use of `substr()` function.
5. Where is string manipulation used in PHP applications?

EXPERIMENT NO. 12

Aim:

To create a PHP script that stores multiple values in an array and displays them on a web page.

Objective:

To understand the concept of arrays in PHP and learn how to store and display multiple values using PHP scripting.

Course Outcomes Addressed:

- Understand PHP data types and arrays
- Apply PHP arrays to store and retrieve multiple values
- Develop dynamic web pages using PHP

Outcome:

Students will successfully create and execute a PHP program that stores multiple values in an array and displays them using PHP syntax.

Software and Hardware Requirement:

- **Operating System:** Windows / Linux
- **Software:** Browser (Chrome/Firefox), VS Code/Notepad++, XAMPP (for PHP)
- **Hardware:** Any system capable of running XAMPP

Theory:

An **array** in PHP is a data structure that allows storing multiple values in a single variable. Each value in an array is accessed using an index or key.

Types of arrays in PHP:

1. **Indexed Array** – Uses numeric index
2. **Associative Array** – Uses named keys
3. **Multidimensional Array** – Array within an array

PHP provides built-in functions like:

- `count()` – Counts number of elements
- `foreach()` – Iterates through array elements

Procedure:

1. Install and start XAMPP server.
2. Open a text editor.
3. Write a PHP script to create and display an array.
4. Save the file with `.php` extension inside `htdocs`.
5. Run the file using `localhost` in a web browser.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>PHP Array Example</title>
</head>
<body>

<h1>Student Subjects</h1>

<?php
$subjects = array("Mathematics", "Physics", "Computer Science", "Electronics", "English");

echo "<h3>Subjects List:</h3>";

foreach ($subjects as $value) {
    echo $value . "<br>";
}
?>

</body>
</html>
```

Output:

A web page displaying multiple values stored in a PHP array.

Conclusion:

Thus, a PHP script was successfully created to store multiple values in an array and display them using a loop.

Viva Questions:

1. What is an array in PHP?
2. How many types of arrays are there in PHP?
3. What is the use of `foreach()` loop?
4. How do you declare an array in PHP?
5. What is the difference between indexed and associative arrays?